

2

Tools of the Trade

Before we embark on our journey across the fascinating landscape of game engine architecture, it is important that we equip ourselves with some basic tools and provisions. In the next two chapters, we will review the software engineering concepts and practices that we will need during our voyage. In Chapter 2, we'll explore the tools used by the majority of professional game engineers. Then in Chapter 3, we'll round out our preparations by reviewing some key topics in the realms of object-oriented programming, design patterns and large-scale C++ programming.

Game development is one of the most demanding and broad areas of software engineering, so believe me, we'll want to be well equipped if we are to safely navigate the sometimes-treacherous terrain we'll be covering. For some readers, the contents of this chapter and the next will be very familiar. However, I encourage you not to skip these chapters entirely. I hope that they will serve as a pleasant refresher; and who knows, you might even pick up a new trick or two.

2.1 Version Control

A *version control* system is a tool that permits multiple users to work on a group of files collectively. It maintains a history of each file so that changes

can be tracked and reverted if necessary. It permits multiple users to modify files—even the same file—simultaneously, without everyone stomping on each other’s work. Version control gets its name from its ability to track the version history of files. It is sometimes called *source control*, because it is primarily used by computer programmers to manage their source code. However, version control can be used for other kinds of files as well. Version control systems are usually best at managing text files, for reasons we will discover below. However, many game studios use a single version control system to manage both source code files (which are text) and game assets like textures, 3D meshes, animations and audio files (which are usually binary).

2.1.1 Why Use Version Control?

Version control is crucial whenever software is developed by a team of multiple engineers. Version control

- provides a central repository from which engineers can share source code;
- keeps a history of the changes made to each source file;
- provides mechanisms allowing specific versions of the code base to be tagged and later retrieved; and
- permits versions of the code to be branched off from the main development line, a feature often used to produce demos or make patches to older versions of the software.

A source control system can be useful even on a single-engineer project. Although its multiuser capabilities won’t be relevant, its other abilities, such as maintaining a history of changes, tagging versions, creating branches for demos and patches, tracking bugs, etc., are still invaluable.

2.1.2 Common Version Control Systems

Here are the most common source control systems you’ll probably encounter during your career as a game engineer.

- *SCCS and RCS*. The Source Code Control System (SCCS) and the Revision Control System (RCS) are two of the oldest version control systems. Both employ a command-line interface. They are prevalent primarily on UNIX platforms.
- *CVS*. The Concurrent Version System (CVS) is a heavy-duty professional-grade command-line-based source control system, originally built on top

of RCS (but now implemented as a stand-alone tool). CVS is prevalent on UNIX systems but is also available on other development platforms such as Microsoft Windows. It is open source and licensed under the Gnu General Public License (GPL). CVSNT (also known as WinCVS) is a native Windows implementation that is based on, and compatible with, CVS.

- *Subversion*. Subversion is an open source version control system aimed at replacing and improving upon CVS. Because it is open source and hence free, it is a great choice for individual projects, student projects and small studios.
- *Git*. This is an open source revision control system that has been used for many venerable projects, including the Linux kernel. In the git development model, the programmer makes changes to files and commits the changes to a branch. The programmer can then merge his changes into any other code branch quickly and easily, because git “knows” how to rewind a sequence of diffs and reapply them onto a new base revision—a process git calls *rebasing*. The net result is a revision control system that is highly efficient and fast when dealing with multiple code branches. Git is a distributed version control system; individual programmers can work locally much of the time, yet they can merge their changes into a shared codebase easily. It’s also very easy to use on a one-person software project, because there’s zero server set-up to worry about. More information on git can be found at <http://git-scm.com/>.
- *Perforce*. Perforce is a professional-grade source control system, with both text-based and GUI interfaces. One of Perforce’s claims to fame is its concept of *change lists*. A change list is a collection of source files that have been modified as a logical unit. Change lists are checked into the repository atomically—either the entire change list is submitted, or none of it is. Perforce is used by many game companies, including *Naughty Dog* and *Electronic Arts*.
- *NxN Alienbrain*. Alienbrain is a powerful and feature-rich source control system designed explicitly for the game industry. Its biggest claim to fame is its support for very large databases containing both text source code files and binary game art assets, with a customizable user interface that can be targeted at specific disciplines such as artists, producers or programmers.
- *ClearCase*. Rational ClearCase is a professional-grade source control system aimed at very large-scale software projects. It is powerful and employs a unique user interface that extends the functionality of Windows

Explorer. I haven't seen ClearCase used much in the game industry, perhaps because it is one of the more expensive version control systems.

- *Microsoft Visual SourceSafe*. SourceSafe is a lightweight source control package that has been used successfully on some game projects.

2.1.3 Overview of Subversion and TortoiseSVN

I have chosen to highlight Subversion in this book for a few reasons. First off, it's free, which is always nice. It works well and is reliable, in my experience. A Subversion central repository is quite easy to set up, and as we'll see, there are already a number of free repository servers out there if you don't want to go to the trouble of setting one up yourself. There are also a number of good Windows and Mac Subversion clients, such as the freely available TortoiseSVN for Windows. So while Subversion may not be the best choice for a large commercial project (I personally prefer Perforce or git for that purpose), I find it perfectly suited to small personal and educational projects. Let's take a look at how to set up and use Subversion on a Microsoft Windows PC development platform. As we do so, we'll review core concepts that apply to virtually any version control system.

Subversion, like most other version control systems, employs a client-server architecture. The server manages a central repository, in which a version-controlled directory hierarchy is stored. Clients connect to the server and request operations, such as checking out the latest version of the directory tree, committing new changes to one or more files, tagging revisions, branching the repository and so on. We won't discuss setting up a server here; we'll assume you have a server, and instead we will focus on setting up and using the client. You can learn how to set up a Subversion server by reading Chapter 6 of [43]. However, you probably will never need to do so, because you can always find free Subversion servers. For example, HelixTeamHub provides Subversion code hosting at <http://info.perforce.com/try-perforce-helix-teamhub-free.html> that's free for projects of 5 users or fewer and up to 1 GB of storage. Beanstalk is another good hosting service, but they do charge a nominal monthly fee.

2.1.4 Setting up a Code Repository

The easiest way to get started with Subversion is to visit the website of HelixTeamHub or a similar SVN hosting service, and set up a Subversion repository. Create an account and you're off to the races. Most hosting sites offer easy-to-follow instructions.

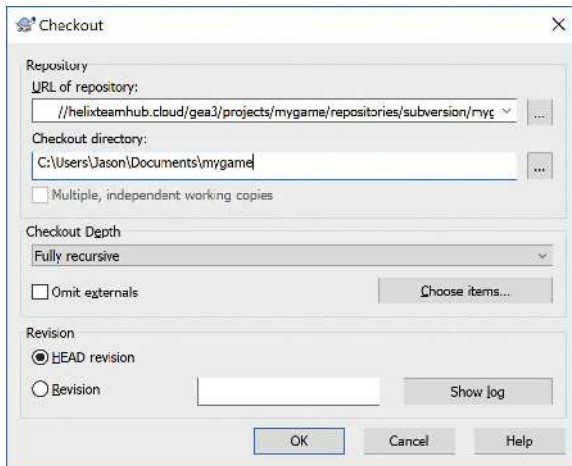


Figure 2.1. TortoiseSVN initial check-out dialog.

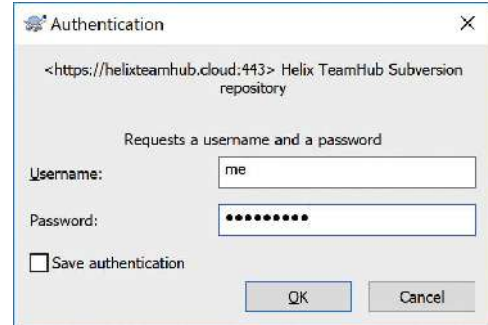


Figure 2.2. TortoiseSVN user authentication dialog.

Once you’ve created your repository, you can typically administer it on the hosting service’s website. You can add and remove users, control options and perform a wealth of advanced tasks. But all you really need to do next is set up a Subversion client and start using your repository.

2.1.5 Installing TortoiseSVN

TortoiseSVN is a popular front end for Subversion. It extends the functionality of the Microsoft Windows Explorer via a convenient right-click menu and overlay icons to show you the status of your version-controlled files and folders.

To get TortoiseSVN, visit <http://tortoisesvn.tigris.org/>. Download the latest version from the download page. Install it by double-clicking the .msi file that you’ve downloaded and following the installation wizard’s instructions.

Once TortoiseSVN is installed, you can go to any folder in Windows Explorer and right-click—TortoiseSVN’s menu extensions should now be visible. To connect to an existing code repository (such as one you created on Helix TeamHub), create a folder on your local hard disk and then right-click and select “SVN Checkout...” The dialog shown in Figure 2.1 will appear. In the “URL of repository” field, enter your repository’s URL. If you are using Helix TeamHub, it would be `https://helixteamhub.cloud/mr3/projects/myprojectname/repositories/subversion/myrepository`, where *myprojectname* is whatever you named your project when you first created it, and *myrepository* is the name of your SVN code repository.

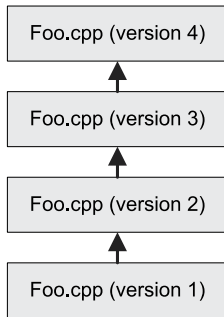


Figure 2.3. File version histories.

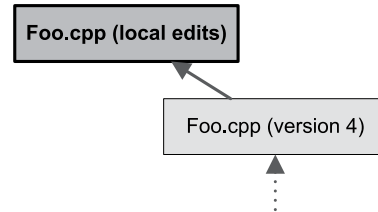
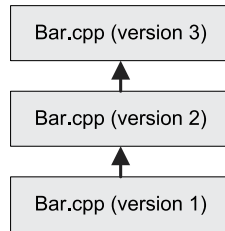


Figure 2.4. Editing the local copy of a version-controlled file.

You should now see the dialog shown in Figure 2.2. Enter your user name and password; checking the “Save authentication” option on this dialog allows you to use your repository without ever having to log in again. Only select this option if you are working on your own personal machine—never on a machine that is shared by many users.

Once you’ve authenticated your user name, TortoiseSVN will download (“check out”) the entire contents of your repository to your local disk. If you have just set up your repository, this will be ...nothing! The folder you created will still be empty. But now it is connected to your Subversion repository on HelixTeamHub (or wherever your server is located). If you refresh your Windows Explorer window (hit F5), you should now see a little green and white checkmark on your folder. This icon indicates that the folder is connected to a Subversion repository via TortoiseSVN and that the local copy of the repository is up to date.

2.1.6 File Versions, Updating and Committing

As we’ve seen, one of the key purposes of any source control system like Subversion is to allow multiple programmers to work on a single software code base by maintaining a central repository or “master” version of all the source code on a server. The server maintains a version history for each file, as shown in Figure 2.3. This feature is crucial to large-scale multiprogrammer software development. For example, if someone makes a mistake and checks in code that “breaks the build,” you can easily go back in time to undo those changes (and check the log to see who the culprit was!). You can also grab a snapshot of the code as it existed at any point in time, allowing you to work with, demonstrate or patch previous versions of the software.

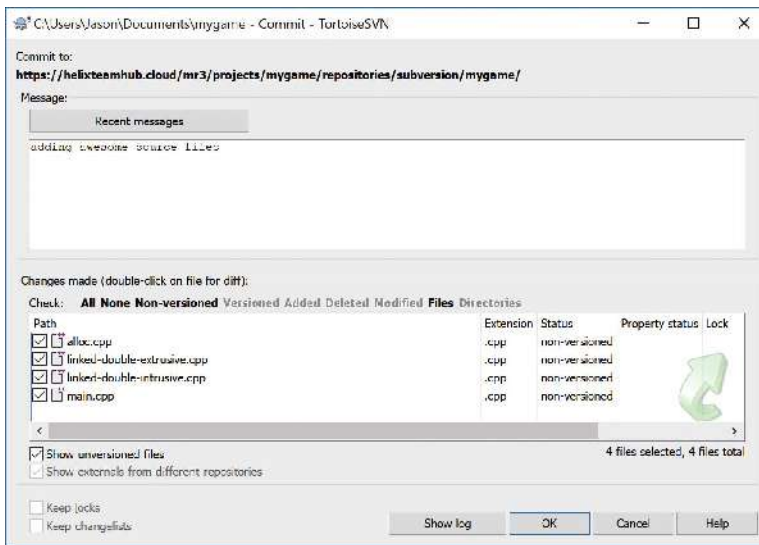


Figure 2.5. TortoiseSVN Commit dialog.

Each programmer gets a local copy of the code on his or her machine. In the case of TortoiseSVN, you obtain your initial working copy by “checking out” the repository, as described above. Periodically you should update your local copy to reflect any changes that may have been made by other programmers. You do this by right-clicking on a folder and selecting “SVN Update” from the pop-up menu.

You can work on your local copy of the code base without affecting the other programmers on the team (Figure 2.4). When you are ready to share your changes with everyone else, you *commit* your changes to the repository (also known as *submitting* or *checking in*). You do this by right-clicking on the folder you want to commit and selecting “SVN Commit...” from the pop-up menu. You will get a dialog like the one shown in Figure 2.5, asking you to confirm the changes.

During a commit operation, Subversion generates a *diff* between your local version of each file and the latest version of that same file in the repository. The term “diff” means difference, and it is typically produced by performing a line-by-line comparison of the two versions of the file. You can double-click on any file in the TortoiseSVN Commit dialog (Figure 2.5) to see the diffs between your version and the latest version on the server (i.e., the changes you made). Files that have changed (i.e., any files that “have diffs”) are committed. This replaces the latest version in the repository with your local version, adding a

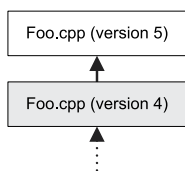


Figure 2.6. Committing local edits to the repository.

new entry to the file’s version history. Any files that have not changed (i.e., your local copy is identical to the latest version in the repository) are ignored by default during a commit. An example commit operation is shown in Figure 2.6.

If you created any new files prior to the commit, they will be listed as “non-versioned” in the Commit dialog. You can check the little check boxes beside them in order to add them to the repository. Any files that you deleted locally will likewise show up as “missing”—if you check their check boxes, they will be deleted from the repository. You can also type a comment in the Commit dialog. This comment is added to the repository’s history log, so that you and others on your team will know why these files were checked in.

2.1.7 Multiple Check-Out, Branching and Merging

Some version control systems require *exclusive check-out*. This means that you must first indicate your intentions to modify a file by *checking it out* and *locking* it. The file(s) that are checked out to you are writable on your local disk and cannot be checked out by anyone else. All other files in the repository are read-only on your local disk. Once you’re done editing the file, you can check it in, which releases the lock and commits the changes to the repository for everyone else to see. The process of exclusively locking files for editing ensures that no two people can edit the same file simultaneously.

Subversion, CVS, Perforce and many other high-quality version control systems also permit *multiple check-out*, i.e., you can edit a file while someone else is editing that same file. Whichever user’s changes are committed first become the latest version of the file in the repository. Any subsequent commits by other users require that programmer to merge his or her changes with the changes made by the programmer(s) who committed previously.

Because more than one set of changes (diffs) have been made to the same file, the version control system must *merge* the changes in order to produce a final version of the file. This is often not a big deal, and in fact many conflicts can be resolved automatically by the version control system. For example, if you changed function `f()` and another programmer changed function `g()`, then your edits would have been to a different range of lines in the file than those of the other programmer. In this case, the merge between your changes and his or her changes will usually resolve automatically without any conflicts. However, if you were both making changes to the same function `f()`, then the second programmer to commit his or her changes will need to do a *three-way merge* (see Figure 2.7).

For three-way merges to work, the version control server has to be smart

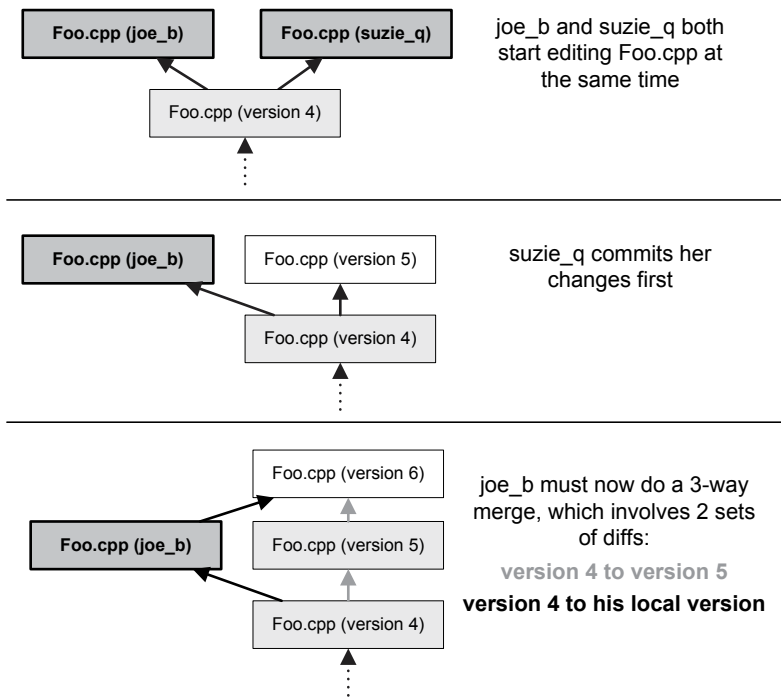


Figure 2.7. Three-way merge due to local edits by two different users.

enough to keep track of which version of each file you currently have on your local disk. That way, when you merge the files, the system will know which version is the base version (the common ancestor, such as version 4 in Figure 2.7).

Subversion permits multiple check-out, and in fact it doesn't require you to check out files explicitly at all. You simply start editing the files locally—all files are writable on your local disk at all times. (By the way, this is one reason that Subversion doesn't scale well to large projects, in my opinion. To determine which files you have changed, Subversion must search the entire tree of source files, which can be slow. Version control systems like Perforce, which explicitly keep track of which files you have modified, are usually easier to work with when dealing with large amounts of code. But for small projects, Subversion's approach works just fine.)

When you perform a commit operation by right-clicking on any folder and selecting "SVN Commit..." from the pop-up menu, you may be prompted to *merge* your changes with changes made by someone else. But if no one has changed the file since you last updated your local copy, then your changes